Cloud-Security-Assignment-4

Q1. Download a sample IAM policy JSON (for example, from AWS documentation), identify all users and their permissions, and list any permissions that are broader than necessary for typical user actions.

Ans:

Step 1: Download a Sample IAM Policy JSON

I referred to the official AWS IAM documentation and selected a sample JSON policy that grants access to a DynamoDB table.

Source: AWS IAM User Guide – Policies and Users

Sample JSON Policy:

```
{
  "Version": "2012-10-17",
  "Statement": {
    "Effect": "Allow",
    "Action": "dynamodb:*",
    "Resource": "arn:aws:dynamodb:us-east-2:123456789012:table/Books"
  }
}
```

Step 2: Identify Users and Their Permissions

The policy does not contain specific IAM usernames. It grants permissions to whichever IAM user, group, or role the policy is attached to.

- **Effect:** Allow permits the specified actions.
- **Action:** dynamodb:* allows all DynamoDB actions.
- **Resource:** The permissions apply to the specified Books table in the given AWS Region and account.
- **Version:** 2012-10-17 specifies the IAM policy language version.

Step 3: Identify Permissions Broader Than Necessary

The permission dynamodb:* is broader than necessary for a typical user who only needs to view table data. It can allow actions such as reading, inserting, updating, and deleting items, as well as other supported operations on the specified table.

For a user who only needs to view data, permissions such as dynamodb:GetItem, dynamodb:Query, and dynamodb:Scan may be more appropriate, depending on the application's requirements.

Result:

The sample policy was reviewed, and dynamodb:* was identified as a broad permission. Following the principle of least privilege, users should receive only the permissions required for their tasks. The policy can be made more secure by replacing the wildcard with specific actions and retaining access only to the required DynamoDB table.

Q2. Edit the given IAM policy to implement least privilege access for a food delivery app scenario (e.g., only allow delivery agents to update order status, not delete orders), and save your revised policy as optimized_policy.json.

Ans:

Optimize IAM Policy for a Food Delivery App

Revised Policy (optimized_policy.json):

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "AllowDeliveryAgentsToUpdateOrderStatusOnly",
   "Effect": "Allow",
   "Action": "dynamodb:UpdateItem",
   "Resource": "arn:aws:dynamodb:ap-south-1:291796569794:table/Orders",
   "Condition": {
    "ForAllValues:StringEquals": {
```

```
        "dynamodb:Attributes": [
          "OrderStatus"
        ]
      },
      "StringEquals": {
        "dynamodb:ReturnValues": "NONE"
      }
    }
  }
 ]
}
```

Explanation: This policy follows the principle of least privilege by allowing delivery agents to update only the OrderStatus attribute in the Orders DynamoDB table. It does not grant permission to delete orders or access other tables.

File Name: optimized_policy.json

Q3. Review an IAM policy that grants full S3 access to a group called 'AppUsers' and modify it so users can only read from a specific bucket named 'user-uploads', not write or delete files. Hint: Use the 'Action' and 'Resource' fields to restrict permissions.

Ans:

1. **Modified IAM Policy (JSON)**

```
{
 "Version": "2012-10-17",
 "Statement": [
   {
    "Sid": "AllowListingUserUploadsBucket",
    "Effect": "Allow",
    "Action": "s3:ListBucket",
    "Resource": "arn:aws:s3:::user-uploads"
   },
```

```
    {
     "Sid": "AllowReadingUserUploadsObjects",
     "Effect": "Allow",
     "Action": "s3:GetObject",
     "Resource": "arn:aws:s3:::user-uploads/*"
    }
   ]
}
```

This policy follows the principle of least privilege by allowing members of the AppUsers group to list and read objects from the user-uploads bucket only. The s3:ListBucket action allows users to view the bucket's contents, while s3:GetObject allows them to download objects. Write and delete permissions are not granted.

Q4. Write a checklist of 5 steps you would follow to regularly review IAM policies for a movie ticket booking app (like BookMyShow), including how you would identify unused permissions and revoke them.

Ans:

Checklist: Regular IAM Policy Review for a Movie Ticket Booking App

- Step 1: Review IAM users, groups, and roles. Check the permissions assigned to employees, developers, booking services, and payment services. Remove access for inactive users.
- Step 2: Check existing IAM policies. Review each policy's Action and Resource fields to identify permissions that are too broad, such as s3:* or *. Apply the principle of least privilege.
- Step 3: Identify unused permissions. Use AWS IAM Access Analyzer and CloudTrail activity logs to check which permissions have not been used. Review the findings carefully before removing any permission.
- Step 4: Revoke unnecessary permissions. Remove unused actions from policies, detach unnecessary policies from users or groups, and replace broad permissions with specific actions. For example, allow a booking service to read movie details without allowing it to delete them.

- Step 5: Test and review regularly. Use the IAM Policy Simulator to test whether required actions are allowed and unauthorized actions are denied. Document the changes and repeat the review regularly.

Conclusion: Regular IAM policy reviews help protect customer information, movie bookings, and payment-related resources by ensuring that users and services have only the permissions they need.

Q5. Use ChatGPT or another AI tool to analyze a provided IAM policy snippet and suggest at least two improvements to enhance security following best practices. Hint: Paste the policy into the AI tool and ask for recommendations based on least privilege and regular review.

Ans:

1. Sample IAM Policy

```
{
 "Version": "2012-10-17",
 "Statement": [
   {
    "Effect": "Allow",
    "Action": "s3:*",
    "Resource": "*"
   }
 ]
}
```

Improvements:

1. Apply the Principle of Least Privilege: Replace "s3:*" with only the required actions, such as s3:ListBucket and s3:GetObject. Replace "Resource": "*" with the ARN of the specific S3 bucket and its objects to prevent unnecessary access.
2. Remove Unused Permissions: Regularly review IAM policies using AWS IAM Access Analyzer and CloudTrail logs. Identify unused permissions and remove them after confirming they are no longer required.

3. Separate Read and Write Permissions: Create separate policies for reading, uploading, and deleting files. Grant users only the permissions necessary for their tasks to reduce the risk of unauthorized changes or data loss.